

Lecture 07 - Visibility Determination

This lecture asks which surfaces are visible from a viewpoint. Rasterization can generate many fragment candidates, but visibility decides which ones should affect the image.

Source chunk	course_text_parts/03_lectures/07_visibility-determination.txt
Extracted slide pages	69
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Visibility Determination
Source cue	- Problem / - Given scene objects and virtual camera / - Determine visible parts of scene objects / - Not occluded by same scene object / - Not occluded by other scene objects

Professor-style explanation	The slide 'Visibility Determination' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Problem / - Given scene objects and virtual camera / - Determine visible parts of scene objects / - Not occluded by same scene object / - Not occluded by other scene objects. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Visibility Determination. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Problem / - Given scene objects and virtual camera / - Determine visible parts of scene objects / - Not occluded by same scene object / - Not occluded by other scene objects
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Visibility Determination' changes positions before rasterization?

Page 3	Object-Based Algorithms
Source cue	- Geometry-based analysis of scene objects before rasterization / - Compare each scene object against all other scene objects / - Determine visible parts based on virtual camera / - Rasterize the visible scene parts / For each object in sceneObjects
Professor-style explanation	The slide 'Object-Based Algorithms' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Geometry-based analysis of scene objects before rasterization / - Compare each scene object against all other scene objects / - Determine visible parts based on virtual camera / - Rasterize the visible scene parts / For each object in sceneObjects. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.

Technical commentary	This slide is about Object-Based Algorithms. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Geometry-based analysis of scene objects before rasterization / - Compare each scene object against all other scene objects / - Determine visible parts based on virtual camera / - Rasterize the visible scene parts / For each object in sceneObjects
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Object-Based Algorithms' changes positions before rasterization?

Page 4	Image-Based Algorithms
Source cue	- Pixel-based analysis of scene objects after rasterization / - Rasterize scene objects based on image resolution / - Investigate for each pixel which scene object is visible / For each pixel in outputRaster / // Determine which scene object is visible at this pixel
Professor-style explanation	The slide 'Image-Based Algorithms' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Pixel-based analysis of scene objects after rasterization / - Rasterize scene objects based on image resolution / - Investigate for each pixel which scene object is visible / For each pixel in outputRaster / // Determine which scene object is visible at this pixel. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Image-Based Algorithms. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Pixel-based analysis of scene objects after rasterization / - Rasterize scene objects based on image resolution / - Investigate for each pixel which scene object is visible / For each pixel in outputRaster / // Determine which scene object is visible at this pixel
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Image-Based Algorithms'?

Page 5	7.1 Object-Based Algorithms
Source cue	7.2 Binary Space Partitioning / 7.3 Warnock Algorithm / 7.4 Depth Buffer Algorithm / 7.5 Depth Buffer Extensions / 7.6 Ray Casting

Professor-style explanation	The slide '7.1 Object-Based Algorithms' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: 7.2 Binary Space Partitioning / 7.3 Warnock Algorithm / 7.4 Depth Buffer Algorithm / 7.5 Depth Buffer Extensions / 7.6 Ray Casting. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about 7.1 Object-Based Algorithms. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: 7.2 Binary Space Partitioning / 7.3 Warnock Algorithm / 7.4 Depth Buffer Algorithm / 7.5 Depth Buffer Extensions / 7.6 Ray Casting
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '7.1 Object-Based Algorithms'?

Page 6	7.1 Object-Based Algorithms
Source cue	Exploiting sorting and clipping
Professor-style explanation	The slide '7.1 Object-Based Algorithms' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: Exploiting sorting and clipping. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about 7.1 Object-Based Algorithms. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: Exploiting sorting and clipping
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in '7.1 Object-Based Algorithms'?

Page 7	Painter's Algorithm 1/3
Source cue	Bob Ross – Grandeur of Summer

Professor-style explanation	The slide 'Painter's Algorithm 1/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: Bob Ross – Grandeur of Summer. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Painter's Algorithm 1/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: Bob Ross – Grandeur of Summer
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Painter's Algorithm 1/3' into a causal sentence instead of repeating the slide title?

Page 8	Painter's Algorithm 2/3
Source cue	- First algorithm for object-based visible polygon determination / - Input: set of polygons / - Output: rendering of visible polygons / - Procedure: simulate drawing process of a painting / 1. Sort polygons into list in back-to-front order
Professor-style explanation	The slide 'Painter's Algorithm 2/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - First algorithm for object-based visible polygon determination / - Input: set of polygons / - Output: rendering of visible polygons / - Procedure: simulate drawing process of a painting / 1. Sort polygons into list in back-to-front order. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Painter's Algorithm 2/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - First algorithm for object-based visible polygon determination / - Input: set of polygons / - Output: rendering of visible polygons / - Procedure: simulate drawing process of a painting / 1. Sort polygons into list in back-to-front order
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Painter's Algorithm 2/3' into a causal sentence instead of repeating the slide title?

Page 9	Painter's Algorithm 3/3
Source cue	- Advantages / - Simple implementation / - No special graphics hardware necessary / - Disadvantages / - Works for polygons only instead of 3D scene objects

Professor-style explanation	The slide 'Painter's Algorithm 3/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Advantages / - Simple implementation / - No special graphics hardware necessary / - Disadvantages / - Works for polygons only instead of 3D scene objects. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Painter's Algorithm 3/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Advantages / - Simple implementation / - No special graphics hardware necessary / - Disadvantages / - Works for polygons only instead of 3D scene objects
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Painter's Algorithm 3/3' into a causal sentence instead of repeating the slide title?

Page 10	Weiler-Atherton Algorithm 1/3
Source cue	- Clipping-based divide-and-conquer approach / - Input: set of polygons / - Output: set of visible polygon parts / - Procedure / 1. Sort polygons into list ■ in front-to-back order
Professor-style explanation	The slide 'Weiler-Atherton Algorithm 1/3' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Clipping-based divide-and-conquer approach / - Input: set of polygons / - Output: set of visible polygon parts / - Procedure / 1. Sort polygons into list ■ in front-to-back order. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Weiler-Atherton Algorithm 1/3. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Clipping-based divide-and-conquer approach / - Input: set of polygons / - Output: set of visible polygon parts / - Procedure / 1. Sort polygons into list ■ in front-to-back order
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Weiler-Atherton Algorithm 1/3'?

Page 11	Weiler-Atherton Algorithm 2/3
---------	-------------------------------

Source cue	- Example from original paper / - Z value order flipped / - Bold: clip polygon / - Gray: unoccluded parts
Professor-style explanation	The slide 'Weiler-Atherton Algorithm 2/3' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Example from original paper / - Z value order flipped / - Bold: clip polygon / - Gray: unoccluded parts. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Weiler-Atherton Algorithm 2/3. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Example from original paper / - Z value order flipped / - Bold: clip polygon / - Gray: unoccluded parts
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Weiler-Atherton Algorithm 2/3'?
Page 12	Weiler-Atherton Algorithm 3/3
Source cue	- Advantages / - Can handle lines and polygons / - Exact calculation of the visible polygons / - Can be used for wireframe rendering / with different edge styles
Professor-style explanation	The slide 'Weiler-Atherton Algorithm 3/3' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Advantages / - Can handle lines and polygons / - Exact calculation of the visible polygons / - Can be used for wireframe rendering / with different edge styles. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Weiler-Atherton Algorithm 3/3. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Advantages / - Can handle lines and polygons / - Exact calculation of the visible polygons / - Can be used for wireframe rendering / with different edge styles
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Weiler-Atherton Algorithm 3/3'?

Page 13	Back Face Culling 1/2
Source cue	- Solves visibility determination for one convex, opaque scene object, / as its back faces are not visible / - Algorithm culls away polygons facing away from virtual camera / - Input: convex, opaque 3D scene object / - Output: set of front-facing polygons of input scene object
Professor-style explanation	The slide 'Back Face Culling 1/2' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Solves visibility determination for one convex, opaque scene object, / as its back faces are not visible / - Algorithm culls away polygons facing away from virtual camera / - Input: convex, opaque 3D scene object / - Output: set of front-facing polygons of input scene object. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Back Face Culling 1/2. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Solves visibility determination for one convex, opaque scene object, / as its back faces are not visible / - Algorithm culls away polygons facing away from virtual camera / - Input: convex, opaque 3D scene object / - Output: set of front-facing polygons of input scene object
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Back Face Culling 1/2' changes positions before rasterization?
Page 14	Back Face Culling 2/2
Source cue	- Advantages / - Simple implementation / - An average of 50% of the areas of a scene can be eliminated in this way / - Reduces number of polygons to be processed, / thus reduces GPU transfers and speeds up the rendering process

Professor-style explanation	The slide 'Back Face Culling 2/2' explains rendering as a chain of object transformations. At the beginning there is scene information: models, vertices, primitives, attributes, camera state, and rendering state. The geometry part of the pipeline changes where the objects are and how they are represented; vertices become positioned data, vertices are assembled into primitives, and primitives are prepared for conversion to the image grid. On the slide, the concrete items are: - Advantages / - Simple implementation / - An average of 50% of the areas of a scene can be eliminated in this way / - Reduces number of polygons to be processed, / thus reduces GPU transfers and speeds up the rendering process. The important story is that the same scene information changes form several times. A triangle is first model data, then transformed geometry, then a projected primitive, then a set of fragments, and finally only some of those fragments become pixel updates. This is why the pipeline is not just a drawing diagram; it is the explanation for where image errors can enter.
Technical commentary	This slide is about Back Face Culling 2/2. Scene or model data moves through a sequence of representations: vertices, primitives, fragments, tests, and framebuffer updates. Each named object is one stage in that conversion. Concrete items shown: - Advantages / - Simple implementation / - An average of 50% of the areas of a scene can be eliminated in this way / - Reduces number of polygons to be processed, / thus reduces GPU transfers and speeds up the rendering process
Why it matters	Pipeline understanding lets you localize rendering errors instead of guessing randomly.
Check yourself	Can you name the input and output representation for 'Back Face Culling 2/2' in the rendering pipeline?
Page 15	Robert's Algorithm
Source cue	- First algorithm for object-based visible line determination / - Input: set of convex 3D scene objects / - Output: set of visible lines / - Procedure: perform for all scene objects the following steps / 1. Remove self-occluded surfaces through back-face culling
Professor-style explanation	The slide 'Robert's Algorithm' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - First algorithm for object-based visible line determination / - Input: set of convex 3D scene objects / - Output: set of visible lines / - Procedure: perform for all scene objects the following steps / 1. Remove self-occluded surfaces through back-face culling. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.

Technical commentary	This slide is about Robert's Algorithm. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - First algorithm for object-based visible line determination / - Input: set of convex 3D scene objects / - Output: set of visible lines / - Procedure: perform for all scene objects the following steps / 1. Remove self-occluded surfaces through back-face culling
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Robert's Algorithm'?

Page 16	Assessment Robert's Algorithm
Source cue	- Advantages / - Determination of visible lines was particularly important / for presentation on vector screens / - Today used in non-photorealistic rendering / to generate line drawings
Professor-style explanation	The slide 'Assessment Robert's Algorithm' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Advantages / - Determination of visible lines was particularly important / for presentation on vector screens / - Today used in non-photorealistic rendering / to generate line drawings. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Assessment Robert's Algorithm. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Advantages / - Determination of visible lines was particularly important / for presentation on vector screens / - Today used in non-photorealistic rendering / to generate line drawings
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Assessment Robert's Algorithm'?

Page 17	7.2 Binary Space Partitioning
Source cue	Hierarchical scene bisection

Professor-style explanation	The slide '7.2 Binary Space Partitioning' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: Hierarchical scene bisection. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about 7.2 Binary Space Partitioning. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: Hierarchical scene bisection
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn '7.2 Binary Space Partitioning' into a causal sentence instead of repeating the slide title?

Page 18	Spatial Data Structures
Source cue	- Spatial data structures are specialized / for organizing and storing scene data / - Types of spatial data structures / - Quad Trees: Optimize 2D data retrieval by dividing space it into four nodes / - Octrees: Optimize 3D data retrieval by dividing space it into eight nodes
Professor-style explanation	The slide 'Spatial Data Structures' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Spatial data structures are specialized / for organizing and storing scene data / - Types of spatial data structures / - Quad Trees: Optimize 2D data retrieval by dividing space it into four nodes / - Octrees: Optimize 3D data retrieval by dividing space it into eight nodes. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Spatial Data Structures. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Spatial data structures are specialized / for organizing and storing scene data / - Types of spatial data structures / - Quad Trees: Optimize 2D data retrieval by dividing space it into four nodes / - Octrees: Optimize 3D data retrieval by dividing space it into eight nodes
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Spatial Data Structures' into a causal sentence instead of repeating the slide title?

Page 19	BSP Trees 1/2
---------	---------------

Source cue	- BSP Trees are Binary Space Partitioning trees / that recursively subdivide space into convex subsets through hyperplanes / - Key functions / - Space organization: divide 3D space into subspace represented as nodes / - Query efficiency: facilitate rapid querying like determining object visibility
Professor-style explanation	The slide 'BSP Trees 1/2' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - BSP Trees are Binary Space Partitioning trees / that recursively subdivide space into convex subsets through hyperplanes / - Key functions / - Space organization: divide 3D space into subspace represented as nodes / - Query efficiency: facilitate rapid querying like determining object visibility. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about BSP Trees 1/2. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - BSP Trees are Binary Space Partitioning trees / that recursively subdivide space into convex subsets through hyperplanes / - Key functions / - Space organization: divide 3D space into subspace represented as nodes / - Query efficiency: facilitate rapid querying like determining object visibility
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'BSP Trees 1/2' works per object, per image region, per ray, or per fragment?
Page 20	BSP Trees 2/2
Source cue	- Definition binary tree / - A binary tree is a tree with at most two child nodes per node / - Different traversal schemes for binary trees exist A X X / - In-order traversal: first left child, then root, then right child / B C Y C
Professor-style explanation	The slide 'BSP Trees 2/2' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Definition binary tree / - A binary tree is a tree with at most two child nodes per node / - Different traversal schemes for binary trees exist A X X / - In-order traversal: first left child, then root, then right child / B C Y C. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.

Technical commentary	This slide is about BSP Trees 2/2. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Definition binary tree / - A binary tree is a tree with at most two child nodes per node / - Different traversal schemes for binary trees exist A X X / - In-order traversal: first left child, then root, then right child / B C Y C
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'BSP Trees 2/2' works per object, per image region, per ray, or per fragment?

Page 21	Node Data Structure
Source cue	- Each node contains / - Hyperplane / - List of scene geometry associated with the node / Class BSPNode: / Declare plane as Plane
Professor-style explanation	The slide 'Node Data Structure' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Each node contains / - Hyperplane / - List of scene geometry associated with the node / Class BSPNode: / Declare plane as Plane. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Node Data Structure. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Each node contains / - Hyperplane / - List of scene geometry associated with the node / Class BSPNode: / Declare plane as Plane
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Node Data Structure' works per object, per image region, per ray, or per fragment?

Page 22	BSP Tree Construction
Source cue	- Choose a hyperplane from scene geometry for root node / and associate that plane with root node, split remaining geometry at plane / - Proceed recursively with the two resulting sets of scene geometries / and check each polygon against hyperplane / - Coinciding: associate with node

Professor-style explanation	The slide 'BSP Tree Construction' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Choose a hyperplane from scene geometry for root node / and associate that plane with root node, split remaining geometry at plane / - Proceed recursively with the two resulting sets of scene geometries / and check each polygon against hyperplane / - Coinciding: associate with node. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about BSP Tree Construction. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Choose a hyperplane from scene geometry for root node / and associate that plane with root node, split remaining geometry at plane / - Proceed recursively with the two resulting sets of scene geometries / and check each polygon against hyperplane / - Coinciding: associate with node
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'BSP Tree Construction' works per object, per image region, per ray, or per fragment?

Page 23	BSP Tree Construction - Example
Source cue	1 2 / 3 4 / A B E / 3 1
Professor-style explanation	The slide 'BSP Tree Construction - Example' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: 1 2 / 3 4 / A B E / 3 1. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about BSP Tree Construction - Example. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: 1 2 / 3 4 / A B E / 3 1
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'BSP Tree Construction - Example' works per object, per image region, per ray, or per fragment?

Page 24	BSP Tree Construction – Pseudo Code
---------	-------------------------------------

Source cue	Function buildBSPTree(BSPNode node, List of Polygons list) // Select one polygon to use as a hyperplane / Polygon p = Get first polygon from list / Set node.plane to the plane of polygon p / Add polygon p to node.geometry
Professor-style explanation	The slide 'BSP Tree Construction – Pseudo Code' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: Function buildBSPTree(BSPNode node, List of Polygons list) // Select one polygon to use as a hyperplane / Polygon p = Get first polygon from list / Set node.plane to the plane of polygon p / Add polygon p to node.geometry. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about BSP Tree Construction – Pseudo Code. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: Function buildBSPTree(BSPNode node, List of Polygons list) // Select one polygon to use as a hyperplane / Polygon p = Get first polygon from list / Set node.plane to the plane of polygon p / Add polygon p to node.geometry
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'BSP Tree Construction – Pseudo Code' works per object, per image region, per ray, or per fragment?
Page 25	Visibility Determination
Source cue	- Visibility determination through modified depth-first traversal / respecting the current camera / - Tree traversal handles the entire scene geometry / - Drawing is done in analogy to painter's algorithm / - Traversing provides the necessary "strict back-to-front order"

Professor-style explanation	The slide 'Visibility Determination' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Visibility determination through modified depth-first traversal / respecting the current camera / - Tree traversal handles the entire scene geometry / - Drawing is done in analogy to painter's algorithm / - Traversing provides the necessary "strict back-to-front order". The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Visibility Determination. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Visibility determination through modified depth-first traversal / respecting the current camera / - Tree traversal handles the entire scene geometry / - Drawing is done in analogy to painter's algorithm / - Traversing provides the necessary "strict back-to-front order"
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Visibility Determination' changes positions before rasterization?

Page 26	Visibility Determination – Pseudo Code
Source cue	Function BSPHiddenSurfaceTraversal(BSPNode node, Point eye) / // Calculate the signed distance from the eye point to the partition plane / float distance = classifyPoint(node.plane, eye) / If distance < 0 Then / // Eye-point is on the back side of the plane
Professor-style explanation	The slide 'Visibility Determination – Pseudo Code' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: Function BSPHiddenSurfaceTraversal(BSPNode node, Point eye) / // Calculate the signed distance from the eye point to the partition plane / float distance = classifyPoint(node.plane, eye) / If distance < 0 Then / // Eye-point is on the back side of the plane. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.

Technical commentary	This slide is about Visibility Determination – Pseudo Code. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: Function BSPHiddenSurfaceTraversal(BSPNode node, Point eye) // Calculate the signed distance from the eye point to the partition plane / float distance = classifyPoint(node.plane, eye) / If distance < 0 Then // Eye-point is on the back side of the plane
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Visibility Determination – Pseudo Code' works per object, per image region, per ray, or per fragment?
Page 27	BSP Tree Traversal - Example
Source cue	■ ■ / 4 1 2 / ■ ■ ■ ■ / 3 4 / ■ ■ ■ ■
Professor-style explanation	The slide 'BSP Tree Traversal - Example' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: ■ ■ / 4 1 2 / ■ ■ ■ ■ / 3 4 / ■ ■ ■ ■. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about BSP Tree Traversal - Example. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: ■ ■ / 4 1 2 / ■ ■ ■ ■ / 3 4 / ■ ■ ■ ■
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'BSP Tree Traversal - Example' works per object, per image region, per ray, or per fragment?
Page 28	Splitting Planes Selection
Source cue	- Hyperplane selection influences tree balance and required geometry splits / - Two alternative approaches exist / - Planes can be derived from scene geometry / - Planes can be chosen freely / - Useful heuristics

Professor-style explanation	The slide 'Splitting Planes Selection' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Hyperplane selection influences tree balance and required geometry splits / - Two alternative approaches exist / - Planes can be derived from scene geometry / - Planes can be chosen freely / - Useful heuristics. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Splitting Planes Selection. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Hyperplane selection influences tree balance and required geometry splits / - Two alternative approaches exist / - Planes can be derived from scene geometry / - Planes can be chosen freely / - Useful heuristics
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Splitting Planes Selection' into a causal sentence instead of repeating the slide title?

Page 29	Traversal Optimization
Source cue	- Sub-trees can be skipped during traversal if content is outside view frustum / - Testing is done by checking 8 corner points of 3D view frustum / - If there are two points on different sides, then the subtree is traversed / - If all points are in one subspace, then the other subspace does not have / to be traversed (view-frustum culling)
Professor-style explanation	The slide 'Traversal Optimization' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Sub-trees can be skipped during traversal if content is outside view frustum / - Testing is done by checking 8 corner points of 3D view frustum / - If there are two points on different sides, then the subtree is traversed / - If all points are in one subspace, then the other subspace does not have / to be traversed (view-frustum culling). The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Traversal Optimization. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Sub-trees can be skipped during traversal if content is outside view frustum / - Testing is done by checking 8 corner points of 3D view frustum / - If there are two points on different sides, then the subtree is traversed / - If all points are in one subspace, then the other subspace does not have / to be traversed (view-frustum culling)

Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Traversal Optimization' works per object, per image region, per ray, or per fragment?
Page 30	Assessment BSP Trees
Source cue	- Advantages / - Simple implementation / - Do not need a depth buffer / - Applicable to static and dynamic scenes / (static scene geometry precalculated, dynamic objects inserted)
Professor-style explanation	The slide 'Assessment BSP Trees' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Advantages / - Simple implementation / - Do not need a depth buffer / - Applicable to static and dynamic scenes / (static scene geometry precalculated, dynamic objects inserted). The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Assessment BSP Trees. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Advantages / - Simple implementation / - Do not need a depth buffer / - Applicable to static and dynamic scenes / (static scene geometry precalculated, dynamic objects inserted)
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Assessment BSP Trees'?
Page 31	7.3 Warnock Algorithm
Source cue	Image-based divide-and-conquer strategy
Professor-style explanation	The slide '7.3 Warnock Algorithm' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: Image-based divide-and-conquer strategy. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.

Technical commentary	This slide is about 7.3 Warnock Algorithm. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: Image-based divide-and-conquer strategy
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether '7.3 Warnock Algorithm' works per object, per image region, per ray, or per fragment?

Page 32	Warnock Algorithm
Source cue	- Hybrid object- and image-based divide-and-conquer approach / - Input: set of polygons and viewport / - Output: set of visible polygon parts / - Procedure: exploit spatial coherence of projected polygons to divide viewport / - Divide viewport recursively into squared regions
Professor-style explanation	The slide 'Warnock Algorithm' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Hybrid object- and image-based divide-and-conquer approach / - Input: set of polygons and viewport / - Output: set of visible polygon parts / - Procedure: exploit spatial coherence of projected polygons to divide viewport / - Divide viewport recursively into squared regions. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Warnock Algorithm. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Hybrid object- and image-based divide-and-conquer approach / - Input: set of polygons and viewport / - Output: set of visible polygon parts / - Procedure: exploit spatial coherence of projected polygons to divide viewport / - Divide viewport recursively into squared regions
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Warnock Algorithm' changes positions before rasterization?
Page 33	Region-Based Visibility Determination
Source cue	■ ■ / Case 1 / - Given viewport region ■ and set of polygons ■, for the following / cases visibility determination can be considered trivial / - Case 1: All polygons in ■ are outside of ■ ■ ■ ■ ■ ■

Professor-style explanation	The slide 'Region-Based Visibility Determination' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: ■ ■ / Case 1 / - Given viewport region ■ and set of polygons ■, for the following / cases visibility determination can be considered trivial / - Case 1: All polygons in ■ are outside of ■ ■ ■ ■ ■. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Region-Based Visibility Determination. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: ■ ■ / Case 1 / - Given viewport region ■ and set of polygons ■, for the following / cases visibility determination can be considered trivial / - Case 1: All polygons in ■ are outside of ■ ■ ■ ■ ■
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Region-Based Visibility Determination' changes positions before rasterization?

Page 34	Procedure
Source cue	- Start with entire viewport region ■ / - Subdivide ■ into four equal subregions / - Terminate if subregion size matches pixel size / - Classify subregions based on trivial cases / - If subregion matches trivial cases: resolve
Professor-style explanation	The slide 'Procedure' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Start with entire viewport region ■ / - Subdivide ■ into four equal subregions / - Terminate if subregion size matches pixel size / - Classify subregions based on trivial cases / - If subregion matches trivial cases: resolve. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.

Technical commentary	This slide is about Procedure. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Start with entire viewport region ■ / - Subdivide ■ into four equal subregions / - Terminate if subregion size matches pixel size / - Classify subregions based on trivial cases / - If subregion matches trivial cases: resolve
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Procedure' changes positions before rasterization?

Page 35	Pseudo Code
Source cue	Procedure warnockHSR(Polygons list, Viewport vp) / If isSimple(list, vp) Then / // Base case: Simple enough scenario to draw directly / drawPolygons(list) / Else
Professor-style explanation	The slide 'Pseudo Code' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: Procedure warnockHSR(Polygons list, Viewport vp) / If isSimple(list, vp) Then / // Base case: Simple enough scenario to draw directly / drawPolygons(list) / Else. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Pseudo Code. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: Procedure warnockHSR(Polygons list, Viewport vp) / If isSimple(list, vp) Then / // Base case: Simple enough scenario to draw directly / drawPolygons(list) / Else
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Pseudo Code' changes positions before rasterization?

Page 36	Assessment Warnock Algorithm
Source cue	- Advantages / - Efficient for large polygons by exploiting spatial coherence / - Easy to implement through recursive programming / - Disadvantages / - Restriction to polygons as primitives

Professor-style explanation	The slide 'Assessment Warnock Algorithm' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Advantages / - Efficient for large polygons by exploiting spatial coherence / - Easy to implement through recursive programming / - Disadvantages / - Restriction to polygons as primitives. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Assessment Warnock Algorithm. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Advantages / - Efficient for large polygons by exploiting spatial coherence / - Easy to implement through recursive programming / - Disadvantages / - Restriction to polygons as primitives
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Assessment Warnock Algorithm' works per object, per image region, per ray, or per fragment?

Page 37	7.4 Depth Buffer Algorithm
Source cue	Exploiting image storage for visibility determination
Professor-style explanation	The slide '7.4 Depth Buffer Algorithm' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: Exploiting image storage for visibility determination. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about 7.4 Depth Buffer Algorithm. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: Exploiting image storage for visibility determination
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '7.4 Depth Buffer Algorithm'?

Page 38	Depth Buffer Algorithm 1/3
---------	----------------------------

Source cue	- Image-based visibility determination algorithm / that exploits extra graphics memory called depth buffer (= z-buffer) / - Depth buffer / - Depth buffer is a 2D raster that contains depth values / - Depth value calculation happens during rasterization
Professor-style explanation	The slide 'Depth Buffer Algorithm 1/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Image-based visibility determination algorithm / that exploits extra graphics memory called depth buffer (= z-buffer) / - Depth buffer / - Depth buffer is a 2D raster that contains depth values / - Depth value calculation happens during rasterization. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Depth Buffer Algorithm 1/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Image-based visibility determination algorithm / that exploits extra graphics memory called depth buffer (= z-buffer) / - Depth buffer / - Depth buffer is a 2D raster that contains depth values / - Depth value calculation happens during rasterization
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Depth Buffer Algorithm 1/3'?

Page 39	Depth Buffer Algorithm 2/3
Source cue	- Depth buffer algorithm follows occlusion culling principle / Function <code>occlusionCulling(List of Polygons scene)</code> / Initialize an <code>OcclusionRepresentation OR</code> / For each object <code>obj</code> in scene / If <code>obj</code> is occluded by <code>OR</code> Then
Professor-style explanation	The slide 'Depth Buffer Algorithm 2/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Depth buffer algorithm follows occlusion culling principle / Function <code>occlusionCulling(List of Polygons scene)</code> / Initialize an <code>OcclusionRepresentation OR</code> / For each object <code>obj</code> in scene / If <code>obj</code> is occluded by <code>OR</code> Then. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about Depth Buffer Algorithm 2/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Depth buffer algorithm follows occlusion culling principle / Function <code>occlusionCulling(List of Polygons scene)</code> / Initialize an <code>OcclusionRepresentation</code> OR / For each object <code>obj</code> in scene / If <code>obj</code> is occluded by OR Then
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Depth Buffer Algorithm 2/3'?

Page 40	Depth Buffer Algorithm 3/3
Source cue	- Depth buffer algorithm performs depth testing for each fragment / - Depth buffer is initialized with background value (e.g., $z = 1$) / - For every scene object / - Rasterize scene object / - For each fragment
Professor-style explanation	The slide 'Depth Buffer Algorithm 3/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Depth buffer algorithm performs depth testing for each fragment / - Depth buffer is initialized with background value (e.g., $z = 1$) / - For every scene object / - Rasterize scene object / - For each fragment. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Depth Buffer Algorithm 3/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Depth buffer algorithm performs depth testing for each fragment / - Depth buffer is initialized with background value (e.g., $z = 1$) / - For every scene object / - Rasterize scene object / - For each fragment
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Depth Buffer Algorithm 3/3'?

Page 41	Pseudo Code – Depth Buffer Test
Source cue	Function <code>zBufferHSR(List of Geometry list)</code> // Initialize z-buffer and color buffer for each pixel on the screen / For <code>x</code> from 0 to Width / For <code>y</code> from 0 to Height / Set <code>depthbuffer[x][y]</code> to 1.0 // Maximum depth value

Professor-style explanation	The slide 'Pseudo Code – Depth Buffer Test' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: Function <code>zBufferHSR(List of Geometry list)</code> // Initialize z-buffer and color buffer for each pixel on the screen / For x from 0 to Width / For y from 0 to Height / Set <code>depthbuffer[x][y]</code> to 1.0 // Maximum depth value. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Pseudo Code – Depth Buffer Test. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: Function <code>zBufferHSR(List of Geometry list)</code> // Initialize z-buffer and color buffer for each pixel on the screen / For x from 0 to Width / For y from 0 to Height / Set <code>depthbuffer[x][y]</code> to 1.0 // Maximum depth value
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Pseudo Code – Depth Buffer Test'?

Page 42	OpenGL Depth Buffer
Source cue	- OpenGL depth buffer: initialization and activation / - Depth values are limited to [0, 1] (0: near-plane, 1: far-plane) / - Depth test must be explicitly activated / - Depth test comparison function can be changed / Function <code>init()</code>
Professor-style explanation	The slide 'OpenGL Depth Buffer' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - OpenGL depth buffer: initialization and activation / - Depth values are limited to [0, 1] (0: near-plane, 1: far-plane) / - Depth test must be explicitly activated / - Depth test comparison function can be changed / Function <code>init()</code> . The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about OpenGL Depth Buffer. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - OpenGL depth buffer: initialization and activation / - Depth values are limited to [0, 1] (0: near-plane, 1: far-plane) / - Depth test must be explicitly activated / - Depth test comparison function can be changed / Function <code>init()</code>
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Depth Buffer'?

Page 43	Depth Buffer Precision
Source cue	- Depth buffer precision is limited by its bit-depth (e.g., 16-bit, 24-bit, 32-bit), / which determines how finely depth can be distinguished / - Depth buffer precision is not linear: denser at near- and sparser at far- / clipping plane $\Rightarrow$ depth conflicts can occur especially at greater distances / Nvidia Blog
Professor-style explanation	The slide 'Depth Buffer Precision' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Depth buffer precision is limited by its bit-depth (e.g., 16-bit, 24-bit, 32-bit), / which determines how finely depth can be distinguished / - Depth buffer precision is not linear: denser at near- and sparser at far- / clipping plane $\Rightarrow$ depth conflicts can occur especially at greater distances / Nvidia Blog. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Depth Buffer Precision. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Depth buffer precision is limited by its bit-depth (e.g., 16-bit, 24-bit, 32-bit), / which determines how finely depth can be distinguished / - Depth buffer precision is not linear: denser at near- and sparser at far- / clipping plane $\Rightarrow$ depth conflicts can occur especially at greater distances / Nvidia Blog
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Depth Buffer Precision'?

Page 44	Z-Fighting 1/2
Source cue	- Z-fighting occurs when two or more objects are very close together in depth / - Depth buffer can't consistently resolve which one is closer / causing flickering or overlapping artifacts in the rendered scene

Professor-style explanation	The slide 'Z-Fighting 1/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Z-fighting occurs when two or more objects are very close together in depth / - Depth buffer can't consistently resolve which one is closer / causing flickering or overlapping artifacts in the rendered scene. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Z-Fighting 1/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Z-fighting occurs when two or more objects are very close together in depth / - Depth buffer can't consistently resolve which one is closer / causing flickering or overlapping artifacts in the rendered scene
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Z-Fighting 1/2'?
Page 45	Z-Fighting 2/2
Source cue	- Mitigation through depth buffer properties / - Use higher precision buffers by switching from 24-bit to 32-bit depth / - Adjust near and far planes: bringing near plane out- and far plane inwards / reduces the depth range, improving effective depth buffer precision / - Mitigating through rendering properties
Professor-style explanation	The slide 'Z-Fighting 2/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Mitigation through depth buffer properties / - Use higher precision buffers by switching from 24-bit to 32-bit depth / - Adjust near and far planes: bringing near plane out- and far plane inwards / reduces the depth range, improving effective depth buffer precision / - Mitigating through rendering properties. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about Z-Fighting 2/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Mitigation through depth buffer properties / - Use higher precision buffers by switching from 24-bit to 32-bit depth / - Adjust near and far planes: bringing near plane out- and far plane inwards / reduces the depth range, improving effective depth buffer precision / - Mitigating through rendering properties
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Z-Fighting 2/2'?

Page 46	Assessment Depth Buffering
Source cue	- Advantages / - Easy implementation / - No presorting of scene objects required / $\Rightarrow$ Object evaluation can be performed in any order / - Not limited to polygonal geometries
Professor-style explanation	The slide 'Assessment Depth Buffering' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Advantages / - Easy implementation / - No presorting of scene objects required / $\Rightarrow$ Object evaluation can be performed in any order / - Not limited to polygonal geometries. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Assessment Depth Buffering. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Advantages / - Easy implementation / - No presorting of scene objects required / $\Rightarrow$ Object evaluation can be performed in any order / - Not limited to polygonal geometries
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Assessment Depth Buffering'?

Page 47	7.5 Depth Buffer Extensions
Source cue	How to render semi-transparent scene objects, and how to generate halos

Professor-style explanation	The slide '7.5 Depth Buffer Extensions' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: How to render semi-transparent scene objects, and how to generate halos. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about 7.5 Depth Buffer Extensions. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: How to render semi-transparent scene objects, and how to generate halos
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '7.5 Depth Buffer Extensions'?
Page 48	Semi-Transparent Objects 1/2
Source cue	- Rendering of semi-transparent objects requires special handling / - Opaque and semi-transparent geometry need to be separated / - Opaque geometry is rendered first, semi-transparent blended on top / Function draw(Vector scene) / // Clear the framebuffer to prepare for new drawing
Professor-style explanation	The slide 'Semi-Transparent Objects 1/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Rendering of semi-transparent objects requires special handling / - Opaque and semi-transparent geometry need to be separated / - Opaque geometry is rendered first, semi-transparent blended on top / Function draw(Vector scene) / // Clear the framebuffer to prepare for new drawing. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about Semi-Transparent Objects 1/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Rendering of semi-transparent objects requires special handling / - Opaque and semi-transparent geometry need to be separated / - Opaque geometry is rendered first, semi-transparent blended on top / Function draw(Vector scene) / // Clear the framebuffer to prepare for new drawing
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Semi-Transparent Objects 1/2'?

Page 49	Semi-Transparent Objects 2/2
Source cue	- Issue: correct transparency without presorting of scene objects not possible
Professor-style explanation	The slide 'Semi-Transparent Objects 2/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Issue: correct transparency without presorting of scene objects not possible. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Semi-Transparent Objects 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Issue: correct transparency without presorting of scene objects not possible
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Semi-Transparent Objects 2/2' into a causal sentence instead of repeating the slide title?

Page 50	Order-Independent Transparency 1/4
Source cue	- Order-independent transparency exploits depth peeling to avoid sorting / - Depth peeling peels away depth layers one-by-one in front-to-back order / - Extract nearest fragments with smallest z values in first rendering pass / - Extract next further fragments in each subsequent rendering pass / - The n th rendering pass delivers the fragments to the n th depth plane

Professor-style explanation	The slide 'Order-Independent Transparency 1/4' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Order-independent transparency exploits depth peeling to avoid sorting / - Depth peeling peels away depth layers one-by-one in front-to-back order / - Extract nearest fragments with smallest z values in first rendering pass / - Extract next further fragments in each subsequent rendering pass / - The ■th rendering pass delivers the fragments to the nth depth plane. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Order-Independent Transparency 1/4. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Order-independent transparency exploits depth peeling to avoid sorting / - Depth peeling peels away depth layers one-by-one in front-to-back order / - Extract nearest fragments with smallest z values in first rendering pass / - Extract next further fragments in each subsequent rendering pass / - The ■th rendering pass delivers the fragments to the nth depth plane
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Order-Independent Transparency 1/4' works per object, per image region, per ray, or per fragment?
Page 51	Order-Independent Transparency 2/4
Source cue	- Illustration of subsequent layers / Layer 0 Layer 1 Layer 2 / 0 depth 1 0 depth 1 0 depth 1
Professor-style explanation	The slide 'Order-Independent Transparency 2/4' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Illustration of subsequent layers / Layer 0 Layer 1 Layer 2 / 0 depth 1 0 depth 1 0 depth 1. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Order-Independent Transparency 2/4. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Illustration of subsequent layers / Layer 0 Layer 1 Layer 2 / 0 depth 1 0 depth 1 0 depth 1

Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Order-Independent Transparency 2/4' works per object, per image region, per ray, or per fragment?
Page 52	Order-Independent Transparency 3/4
Source cue	- 1st pass / - Render scene with conventional depth test (GL_LESS) / - Store resulting depth layer in Z-Buffer A / - Store resulting color layer as Layer 0 / - 2nd pass
Professor-style explanation	The slide 'Order-Independent Transparency 3/4' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - 1st pass / - Render scene with conventional depth test (GL_LESS) / - Store resulting depth layer in Z-Buffer A / - Store resulting color layer as Layer 0 / - 2nd pass. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Order-Independent Transparency 3/4. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - 1st pass / - Render scene with conventional depth test (GL_LESS) / - Store resulting depth layer in Z-Buffer A / - Store resulting color layer as Layer 0 / - 2nd pass
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Order-Independent Transparency 3/4'?
Page 53	Order-Independent Transparency 4/4
Source cue	1 layer 2 layers / 3 layers 4 layers
Professor-style explanation	The slide 'Order-Independent Transparency 4/4' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: 1 layer 2 layers / 3 layers 4 layers. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Order-Independent Transparency 4/4. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: 1 layer 2 layers / 3 layers 4 layers

Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Order-Independent Transparency 4/4' into a causal sentence instead of repeating the slide title?
Page 54	Generating Haloes 1/2
Source cue	- Halo: circle of light around sun or moon caused by ice crystals in the air; [...]; / a distinguishing zone surrounding a central object / - Haloes as wireframe illustration tools result / in better spatial comprehension / - Add highlights to foreground lines
Professor-style explanation	The slide 'Generating Haloes 1/2' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Halo: circle of light around sun or moon caused by ice crystals in the air; [...]; / a distinguishing zone surrounding a central object / - Haloes as wireframe illustration tools result / in better spatial comprehension / - Add highlights to foreground lines. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Generating Haloes 1/2. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Halo: circle of light around sun or moon caused by ice crystals in the air; [...]; / a distinguishing zone surrounding a central object / - Haloes as wireframe illustration tools result / in better spatial comprehension / - Add highlights to foreground lines
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Generating Haloes 1/2'?
Page 55	Generating Haloes 2/2
Source cue	- Proceeding / - Render wireframe model with thick lines into depth buffer / without writing to the color buffer $\Rightarrow$ depth image contains thick lines / - Render wireframe model with thin lines into color buffer / $\Rightarrow$ Lines of foreground surfaces have a "safety margin"

Professor-style explanation	The slide 'Generating Haloes 2/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Proceeding / - Render wireframe model with thick lines into depth buffer / without writing to the color buffer $\Rightarrow$ depth image contains thick lines / - Render wireframe model with thin lines into color buffer / $\Rightarrow$ Lines of foreground surfaces have a "safety margin". The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Generating Haloes 2/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Proceeding / - Render wireframe model with thick lines into depth buffer / without writing to the color buffer $\Rightarrow$ depth image contains thick lines / - Render wireframe model with thin lines into color buffer / $\Rightarrow$ Lines of foreground surfaces have a "safety margin"
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Generating Haloes 2/2'?

Page 56	7.6 Ray Casting
Source cue	Shooting rays to determine visibility
Professor-style explanation	The slide '7.6 Ray Casting' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: Shooting rays to determine visibility. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about 7.6 Ray Casting. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: Shooting rays to determine visibility
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether '7.6 Ray Casting' works per object, per image region, per ray, or per fragment?

Page 57	Ray Casting Principle
---------	-----------------------

Source cue	- Image-based visibility determination algorithm exploiting ray intersections / - Ray initialization - initialize ray from viewpoint through each pixel / (ray generation) / - Intersection calculation - find object intersection closest to camera / (ray intersection)
Professor-style explanation	The slide 'Ray Casting Principle' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Image-based visibility determination algorithm exploiting ray intersections / - Ray initialization - initialize ray from viewpoint through each pixel / (ray generation) / - Intersection calculation - find object intersection closest to camera / (ray intersection). The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Ray Casting Principle. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Image-based visibility determination algorithm exploiting ray intersections / - Ray initialization - initialize ray from viewpoint through each pixel / (ray generation) / - Intersection calculation - find object intersection closest to camera / (ray intersection)
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Ray Casting Principle' changes positions before rasterization?
Page 58	Ray Initialization 1/3
Source cue	- Ray is defined by origin and direction / - Mathematically parameterized as a 3D line via $\mathbf{r} = \mathbf{o} + \mathbf{d}t$ / - $\mathbf{r} = \mathbf{o} + \mathbf{d}t$
Professor-style explanation	The slide 'Ray Initialization 1/3' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Ray is defined by origin and direction / - Mathematically parameterized as a 3D line via $\mathbf{r} = \mathbf{o} + \mathbf{d}t$ / - $\mathbf{r} = \mathbf{o} + \mathbf{d}t$. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.

Technical commentary	This slide is about Ray Initialization 1/3. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Ray is defined by origin and direction / - Mathematically parameterized as a 3D line via $\mathbf{r} = \mathbf{o} + \mathbf{d}t$ / - $\mathbf{r} = \mathbf{o} + \mathbf{d}t$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Ray Initialization 1/3'?

Page 59	Ray Initialization 2/3
Source cue	- How can we find $\mathbf{r}$? / - Assumption: we have a simplified camera model / - Positioned in $\mathbf{o}$ / - Orthonormal base $\{\mathbf{u}, \mathbf{v}, \mathbf{w}\}$ / - Orthonormal: orthogonal + normalized
Professor-style explanation	The slide 'Ray Initialization 2/3' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - How can we find $\mathbf{r}$? / - Assumption: we have a simplified camera model / - Positioned in $\mathbf{o}$ / - Orthonormal base $\{\mathbf{u}, \mathbf{v}, \mathbf{w}\}$ / - Orthonormal: orthogonal + normalized. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Ray Initialization 2/3. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - How can we find $\mathbf{r}$? / - Assumption: we have a simplified camera model / - Positioned in $\mathbf{o}$ / - Orthonormal base $\{\mathbf{u}, \mathbf{v}, \mathbf{w}\}$ / - Orthonormal: orthogonal + normalized
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Ray Initialization 2/3' changes positions before rasterization?

Page 60	Ray Initialization 3/3
Source cue	- Definition of the pixel grid with $\mathbf{u} \times \mathbf{v}$ pixels / x y / - Size of the pixel raster is $\mathbf{u} - \mathbf{u}_0 \times \mathbf{v} - \mathbf{v}_0$ / - Pixel spacing is / - x-coordinate: $(\mathbf{u} - \mathbf{u}_0)/\mathbf{u}$

Professor-style explanation	The slide 'Ray Initialization 3/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Definition of the pixel grid with $\text{width} \times \text{height}$ pixels / x y / - Size of the pixel raster is $\text{width} - \text{height} \times \text{width} - \text{height}$ / - Pixel spacing is / - x-coordinate: $(\text{width} - \text{height})/\text{width}$. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Ray Initialization 3/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Definition of the pixel grid with $\text{width} \times \text{height}$ pixels / x y / - Size of the pixel raster is $\text{width} - \text{height} \times \text{width} - \text{height}$ / - Pixel spacing is / - x-coordinate: $(\text{width} - \text{height})/\text{width}$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Ray Initialization 3/3'?

Page 61	Intersection Calculation
Source cue	- Find smallest positive t for which $\text{ray}(t)$ is an intersection / - Simple procedure / - Iterate over all scene objects and calculate intersections / - Sort intersections / - Select intersection with smallest t
Professor-style explanation	The slide 'Intersection Calculation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Find smallest positive t for which $\text{ray}(t)$ is an intersection / - Simple procedure / - Iterate over all scene objects and calculate intersections / - Sort intersections / - Select intersection with smallest t . The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Intersection Calculation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Find smallest positive t for which $\text{ray}(t)$ is an intersection / - Simple procedure / - Iterate over all scene objects and calculate intersections / - Sort intersections / - Select intersection with smallest t
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Intersection Calculation' into a causal sentence instead of repeating the slide title?

Page 62	Implicit Surface Intersection Calculation
Source cue	- Find set of all points with $\mathbf{x}, y, z = 0$ / - Examples: plane, sphere, quadrics, ... / $\mathbf{p} = (\mathbf{x}, \mathbf{y}, \mathbf{z})$ / $\mathbf{p} = (x, y, z)$ / $\mathbf{p}$
Professor-style explanation	The slide 'Implicit Surface Intersection Calculation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Find set of all points with $\mathbf{x}, y, z = 0$ / - Examples: plane, sphere, quadrics, ... / $\mathbf{p} = (\mathbf{x}, \mathbf{y}, \mathbf{z})$ / $\mathbf{p} = (x, y, z)$ / $\mathbf{p}$. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Implicit Surface Intersection Calculation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Find set of all points with $\mathbf{x}, y, z = 0$ / - Examples: plane, sphere, quadrics, ... / $\mathbf{p} = (\mathbf{x}, \mathbf{y}, \mathbf{z})$ / $\mathbf{p} = (x, y, z)$ / $\mathbf{p}$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Implicit Surface Intersection Calculation' into a causal sentence instead of repeating the slide title?
Page 63	Intersection Calculation - Triangle
Source cue	- Idea: find intersection with plane which embeds triangle, / and test whether intersection lies inside triangle / 1. Calculate intersection with plane / through implicit surface intersection calculation / 2. Exploit barycentric coordinates
Professor-style explanation	The slide 'Intersection Calculation - Triangle' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Idea: find intersection with plane which embeds triangle, / and test whether intersection lies inside triangle / 1. Calculate intersection with plane / through implicit surface intersection calculation / 2. Exploit barycentric coordinates. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Intersection Calculation - Triangle. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Idea: find intersection with plane which embeds triangle, / and test whether intersection lies inside triangle / 1. Calculate intersection with plane / through implicit surface intersection calculation / 2. Exploit barycentric coordinates

Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Intersection Calculation - Triangle'?
Page 64	Pseudo Code – Analytic Intersection Computation
Source cue	Function castRay(List sceneObjects, Window win, Camera cam) / // Iterate over each pixel on the window / For x from 0 to win.width / For y from 0 to win.height / Initialize ray r = calculateRay(x, y, win, cam) // Calculate ray from camera through pixel (x, y)
Professor-style explanation	The slide 'Pseudo Code – Analytic Intersection Computation' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: Function castRay(List sceneObjects, Window win, Camera cam) / // Iterate over each pixel on the window / For x from 0 to win.width / For y from 0 to win.height / Initialize ray r = calculateRay(x, y, win, cam) // Calculate ray from camera through pixel (x, y). The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Pseudo Code – Analytic Intersection Computation. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: Function castRay(List sceneObjects, Window win, Camera cam) / // Iterate over each pixel on the window / For x from 0 to win.width / For y from 0 to win.height / Initialize ray r = calculateRay(x, y, win, cam) // Calculate ray from camera through pixel (x, y)
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Pseudo Code – Analytic Intersection Computation' changes positions before rasterization?
Page 65	Pseudo Code - Iterative Intersection Calculation (Ray Marching)
Source cue	- Ray march in ■ steps and check all scene objects for intersection / Function intersect(const Ray& ray, const List of SceneObjects& sceneObjects) const / Initialize t as 0.0 // Starting point for ray parameter / Initialize epsilon as 0.01 // Small increment for ray parameter / // Loop to incrementally trace the ray through the scene

Professor-style explanation	The slide 'Pseudo Code - Iterative Intersection Calculation (Ray Marching)' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Ray march in ■ steps and check all scene objects for intersection / Function intersect(const Ray& ray, const List of SceneObjects& sceneObjects) const / Initialize t as 0.0 // Starting point for ray parameter / Initialize epsilon as 0.01 // Small increment for ray parameter / // Loop to incrementally trace the ray through the scene. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Pseudo Code - Iterative Intersection Calculation (Ray Marching). The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Ray march in ■ steps and check all scene objects for intersection / Function intersect(const Ray& ray, const List of SceneObjects& sceneObjects) const / Initialize t as 0.0 // Starting point for ray parameter / Initialize epsilon as 0.01 // Small increment for ray parameter / // Loop to incrementally trace the ray through the scene
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Pseudo Code - Iterative Intersection Calculation (Ray Marching)' works per object, per image region, per ray, or per fragment?

Page 66	Assessment Ray Casting
Source cue	- Advantages / - Image-precise algorithm that support all types of scene objects / - Analytic intersection calculation possible / - Can be parallelized and supported by graphics hardware / - Disadvantages
Professor-style explanation	The slide 'Assessment Ray Casting' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Advantages / - Image-precise algorithm that support all types of scene objects / - Analytic intersection calculation possible / - Can be parallelized and supported by graphics hardware / - Disadvantages. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.

Technical commentary	This slide is about Assessment Ray Casting. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Advantages / - Image-precise algorithm that support all types of scene objects / - Analytic intersection calculation possible / - Can be parallelized and supported by graphics hardware / - Disadvantages
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Assessment Ray Casting' works per object, per image region, per ray, or per fragment?

Page 67	but are crucial for accurate visibility determination
Source cue	- Visibility determination ensures only scene objects visible from camera are rendered / - Effective visibility determination is crucial for rendering efficiency and visual accuracy / - Object-based algorithms perform geometry-based analysis prior to rasterization / - Examples include the Painter's Algorithm, Weiler-Atherton, and Back Face Culling / - These methods often suffer from higher complexity and can be poorly parallelizable
Professor-style explanation	The slide 'but are crucial for accurate visibility determination' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Visibility determination ensures only scene objects visible from camera are rendered / - Effective visibility determination is crucial for rendering efficiency and visual accuracy / - Object-based algorithms perform geometry-based analysis prior to rasterization / - Examples include the Painter's Algorithm, Weiler-Atherton, and Back Face Culling / - These methods often suffer from higher complexity and can be poorly parallelizable. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about but are crucial for accurate visibility determination. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Visibility determination ensures only scene objects visible from camera are rendered / - Effective visibility determination is crucial for rendering efficiency and visual accuracy / - Object-based algorithms perform geometry-based analysis prior to rasterization / - Examples include the Painter's Algorithm, Weiler-Atherton, and Back Face Culling / - These methods often suffer from higher complexity and can be poorly parallelizable
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.

Check yourself	Can you explain how 'but are crucial for accurate visibility determination' changes positions before rasterization?
Page 68	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Literature and other sources used in this chapter. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Literature and other sources used in this chapter' into a causal sentence instead of repeating the slide title?
Page 69	Practice, Addison-Wesley.
Source cue	- Text Books / - Foley J., van Dam, A., Feiner, S. (2013). Computer Graphics: Principles and / Practice, Addison-Wesley. / - Papers / - Everitt, C. (2001). Interactive order-independent transparency. White paper,
Professor-style explanation	The slide 'Practice, Addison-Wesley.' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Text Books / - Foley J., van Dam, A., Feiner, S. (2013). Computer Graphics: Principles and / Practice, Addison-Wesley. / - Papers / - Everitt, C. (2001). Interactive order-independent transparency. White paper,. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Practice, Addison-Wesley.. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Text Books / - Foley J., van Dam, A., Feiner, S. (2013). Computer Graphics: Principles and / Practice, Addison-Wesley. / - Papers / - Everitt, C. (2001). Interactive order-independent transparency. White paper,
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.

07.1 Object-Based Algorithms

Source location: Section 7.1

Commentary

Object-based visibility algorithms reason about geometry before or while comparing surfaces. They can sort, split, or reject objects based on spatial relations. This is different from simply letting every fragment fight in the depth buffer.

The value of object-based reasoning is reducing work or establishing correct order. The difficulty is that geometry can overlap in complicated ways, so simple global ordering is not always possible.

Mental model

Object-based visibility tries to solve visibility with geometry before final pixels are written.

Check yourself

Why can intersecting polygons make simple depth sorting fail?

07.2 Binary Space Partitioning

Source location: Section 7.2

Commentary

A BSP tree recursively divides space with planes. Once built, it can help traverse geometry in a view-dependent order. This is useful for visibility and ordering because the tree encodes spatial relationships.

The cost is preprocessing and possible splitting of geometry. BSP is a good example of trading memory and setup work for faster or more structured visibility decisions later.

Mental model

A BSP tree stores a recursive answer to 'which side of this plane is the geometry on?'

Check yourself

Why can building a BSP tree require splitting polygons?

07.3 Warnock Algorithm

Source location: Section 7.3

Commentary

Warnock's algorithm works in image space by subdividing regions until visibility is simple enough to decide. If a region is ambiguous, split it. If it is simple, fill it.

This illustrates a general graphics strategy: recursively reduce a hard global problem into smaller local problems. The algorithm is less central in modern OpenGL practice than the depth buffer, but it helps contrast

image-space and object-space approaches.

Mental model

When a region is too complicated, subdivide until the answer becomes simple.

Check yourself

What makes Warnock's algorithm image-space rather than object-space?

07.4 Depth Buffer Algorithm

Source location: Section 7.4

Commentary

The depth buffer stores the closest accepted depth at each sample or pixel. For each fragment, compare its depth to the stored depth. If it passes, update the color and depth; otherwise discard it.

The strength of the depth buffer is simplicity and hardware efficiency. The weakness is precision: depth values are finite, and projection distributes precision unevenly. Bad near/far settings can create z-fighting.

Mental model

The depth buffer is a per-sample competition for closest visible fragment.

Check yourself

Why should the near plane not be unnecessarily close to the camera?

07.5 Depth Extensions and Ray Casting

Source location: Sections 7.5-7.6

Commentary

Depth buffer extensions refine or adapt depth-based visibility, for example by handling precision, transparency, or multiple layers more carefully. Standard depth testing alone is not a complete solution for every visibility situation.

Ray casting takes the opposite direction from rasterization: for each image sample, cast a ray into the scene and find the closest intersection. This naturally solves primary visibility but requires intersection work instead of raster coverage work.

Mental model

Rasterization pushes primitives to pixels; ray casting pulls visibility from pixels into the scene.

Check yourself

Why is transparency harder than opaque nearest-surface visibility?

End-of-lecture summary

If you remember only one thing from Lecture 07, remember this: This lecture asks which surfaces are visible from a viewpoint. Rasterization can generate many fragment candidates, but visibility decides which ones

should affect the image.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.